

Advanced Algorithms: Lecture 7 Notes

Dana Randall Spring 2024

Sarthak Mohanty

Aadit Trivedi

Randomized Min-cut

Given some connected graph $G = (V, E)$, a **cut** partitions the vertices into two sets: $\mathcal{S}$ and $\mathcal{T}$.

One of the motivating questions here is what is the minimum # of edges whose removal disconnects the graph?

We will not show why here, but one reason why the minimum-cut problem is of interest is that it is tightly connected to the maximum-flow problem. By the max-flow min-cut theorem, the value of the maximum flow from a source s to a sink t is equal to the capacity of the minimum s - t cut.

Thus, one natural algorithm for solving the global min-cut problem is as follows: Pick some source vertex s , then for every other vertex $t \neq s$, compute the minimum s - t cut. Choose the minimum of all these values.

However, this requires $\mathcal{O}(n)$ calls to the max-flow algorithm. Can we do better? In fact, we can! We will now discuss Karger's algorithm, a randomized algorithm for computing the min-cut.

The algorithm is as follows.

- Repeat until two supernodes remain.
 - Choose an edge at random $e = (u, v)$.
 - Contract this edge, making $[uv]$ a supernode. Remove any self loops and keep all multi-edges.

Let us note some observations.

- Note that there can be potentially many min cuts. An example of a graph that has a unique min cut, on the other hand, is two connected graphs connected with a single edge.
- The min cut size is at most the minimum degree.

This algorithm always returns some cut. But now, we propose the question: Is this cut a minimum cut? We argue that this algorithm gives us a minimum cut with some nontrivial probability.

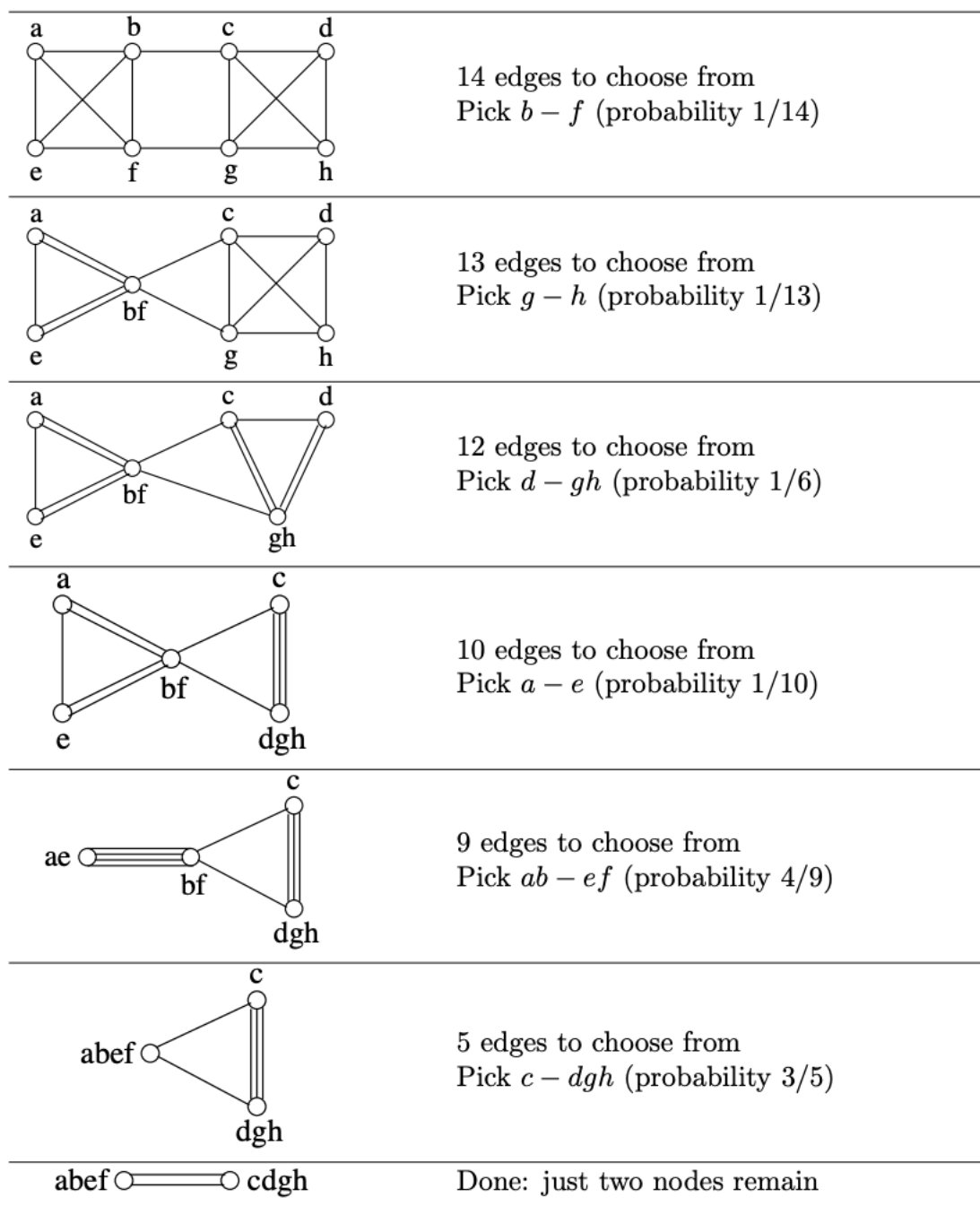


Figure 1: Karger's Algorithm at work (taken from [here](#).)

So, what's the probability that we succeed in finding the min cut with this randomized algorithm? Fix some min cut C of size k . The only way this fixed cut is destroyed is if we contract one of the k edges crossing C .

Since the min cut has size k , every vertex has degree at least k . Thus, when there are n vertices remaining, there are at least $\frac{kn}{2}$ edges remaining.

What is the probability that C survives the first contraction? This is equal to $1 - \text{P}(\text{Failure})$. The probability of failure is at most

$$\frac{k}{kn/2} = \frac{2}{n}.$$

Thus, the probability that C survives the first contraction is at least

$$1 - \frac{2}{n}.$$

More generally, when there are i supernodes remaining, the contracted graph still has min cut at least k , as long as we have not contracted an edge of C . Therefore, every supernode has degree at least k , so there are at least $\frac{ki}{2}$ edges remaining. Thus, the probability that the next contraction destroys C is at most

$$\frac{k}{ki/2} = \frac{2}{i}.$$

Finally, we can calculate the probability that C survives every contraction, as follows:

$$\begin{aligned} \text{P}(C \text{ survives}) &\geq \prod_{i=3}^n \left(1 - \frac{2}{i}\right) \\ &= \prod_{i=3}^n \frac{i-2}{i} \\ &= \frac{1}{3} \cdot \frac{2}{4} \cdot \frac{3}{5} \cdots \frac{n-2}{n} \\ &= \frac{2}{n(n-1)}. \end{aligned}$$

Thus, one run of Karger's algorithm outputs a fixed minimum cut with probability at least

$$\frac{2}{n(n-1)} = \frac{1}{\binom{n}{2}}.$$

This also implies that an n -vertex graph has at most $\binom{n}{2}$ distinct minimum cuts. Indeed, Karger's algorithm outputs exactly one cut, and every minimum cut is output with probability at least $\frac{1}{\binom{n}{2}}$. Therefore, there can be at most $\binom{n}{2}$ minimum cuts.

Boosting the probability of success

Let

$$p_0 \geq \frac{2}{n(n-1)}$$

be the probability that one run of Karger's algorithm succeeds. If we run the algorithm independently N times and keep the smallest cut found, then the probability of success is

$$P^* = 1 - (1 - p_0)^N.$$

Therefore,

$$P^* \geq 1 - \left(1 - \frac{2}{n(n-1)}\right)^N.$$

We use the approximation

$$\lim_{n \rightarrow \infty} \left(1 + \frac{\alpha}{f(n)}\right)^{f(n)} = e^\alpha$$

as long as $\lim_{n \rightarrow \infty} f(n) = \infty$.

Now we have:

$$\begin{aligned} N = n(n-1), & \quad P^* \approx 1 - e^{-2}, \\ N = n(n-1) \ln(n), & \quad P^* \approx 1 - e^{-2 \ln(n)} = 1 - n^{-2}, \\ N = cn(n-1) \ln(n), & \quad P^* \approx 1 - e^{-2c \ln(n)} = 1 - n^{-2c}. \end{aligned}$$

Thus, if we repeat the algorithm $cn(n-1) \ln(n)$ times, then the probability of failure is at most approximately n^{-2c} .

Runtime: Using a straightforward implementation, one run takes $O(mn)$ time. Therefore, after

$$N = O(n^2 \log n)$$

independent runs, the total runtime is

$$O(Nmn) = O(mn^3 \log n).$$

Karger-Stein computation

The Karger-Stein algorithm is an algorithm that computes the minimum cut of a graph with high probability. It improves the original Karger's algorithm by using a recursive approach.

The key idea is that early contractions are relatively safe, while later contractions are much more dangerous. So instead of contracting all the way down to 2 vertices, we contract only down to roughly $\frac{n}{\sqrt{2}}$ vertices and then recurse twice.

Algorithm

The algorithm is as follows:

- (a) If the graph has at most some constant number of vertices, compute the minimum cut directly.
- (b) Let

$$t = \left\lceil \frac{|V|}{\sqrt{2}} + 1 \right\rceil.$$
- (c) Independently run random contraction twice, each time contracting G down to t vertices. Call the resulting graphs G_1 and G_2 .
- (d) Recursively apply the algorithm to G_1 and G_2 .
- (e) Return the smaller of the two cuts found.

Computation

The improvement of the Karger-Stein algorithm comes from its recursive approach. Instead of running one contraction process all the way to 2 vertices, it branches into two recursive calls after contracting to about $\frac{n}{\sqrt{2}}$ vertices.

The runtime recurrence is

$$T(n) = 2T\left(\left\lceil \frac{n}{\sqrt{2}} \right\rceil + 1\right) + O(n^2), \tag{1}$$

where the $O(n^2)$ term accounts for the random contractions used to reduce the graph size.

To solve this recurrence, note that each recursive call has size roughly $\frac{n}{\sqrt{2}}$, and there are two recursive calls. Thus, each level of recursion costs

$$O(n^2),$$

since

$$2 \left(\frac{n}{\sqrt{2}} \right)^2 = n^2.$$

The depth of recursion is $O(\log n)$, so the total runtime is

$$T(n) = O(n^2 \log n).$$

This is a significant improvement over the simple repeated version of Karger's algorithm, which takes $O(mn^3 \log n)$ time using a straightforward implementation.